

```

/**
 * User Identity and Persistence Model.
 *
 * Orchestrates the lifecycle of User entities within the local database.
 * This module manages cryptographic security (hashing/salting), account state
 * transitions, and complex multi-table lookups for roles and access levels.
 *
 * @module User-Persistence-Model
 * @copyright 2018-2026, Dennis Jorgenson
 */

"use strict";

import type { TAccess, IAccess } from "#db/interfaces/state";
import type { ILogger } from "#lib/log.util.js";

import { ApiError, type TResponse } from "#db/types";
import { Select, Insert, Update, Role, State } from "#db/query.util";
import { hashKey } from "#lib/crypto.util";
import { createHmac } from "node:crypto";
import { hasValues, isEqual } from "#lib/std.util";

/**
 * Defines the comprehensive User entity structure.
 */
export interface IUser {
  /** Unique binary identifier for the user. */
  user: Uint8Array;
  /** Unique login name. */
  username: string;
  /** Primary contact email address. */
  email: string;
  /** Binary key representing the user's assigned role. */
  role: Uint8Array;
  /** Human-readable title of the role. */
  title: string;
  /** Numeric authorized privilege ranking. Not all admins can see all accounts */
  auth_rank: number;
  /** Binary key representing the user's current account state. */
  state: Uint8Array;
  /** The current access status (e.g., 'Enabled', 'Disabled'). */
  status: TAccess;
  /** URL path to the user's profile avatar. */
  image_url: string;
  /** Encrypted password storage (Buffer/Uint8Array). */
  password: string | Uint8Array;
  /** The unique 32-byte cryptographic salt used for password hashing. */
  hash: Uint8Array;
  /** Timestamp of account creation. */
  create_time: Date;
  /** Timestamp of the last account modification. */
  update_time: Date;
}

/**
 * Generates a SHA-256 hash of a User object for identity validation or password hashing.
 *
 * @function hashPassword
 * @param {Partial<User.IUser>} props - The User object or properties to be hashed.
 * @returns {Uint8Array} A 32-byte binary Buffer (derived from the 64-character hex hash).
 *
 * @description
 * 1. Serializes the provided user properties into a JSON string.
 * 2. Computes a SHA-256 hash using the native Node.js `crypto` module.
 * 3. Iterates through the resulting 64-character hex string to convert it into a
 *    byte-aligned {@link Uint8Array}.
 * 4. Returns a {@link Buffer} instance, making it compatible with database binary fields.
 *
 * @example
 * const binaryHash = hashPassword({ email: "dennis@example.com", password: "..." });
 */

/**
 * DATABASE PASSWORD HASH (Internal Security)
 * Uses the local node's USER_SECRET to verify identity.
 */
export const hashPassword = (props: any): Buffer | undefined => {
  const { username, email, password } = props;

  // Use the .env secret as the primary pepper
  const salt = process.env.USER_SECRET;
  if (!salt || !password) return undefined;

  // Normalize to prevent "Djorgenson" vs "djorgenson" logic fails
  const identityKey = `${username.toLowerCase().trim()}.${email.toLowerCase().trim()}.${salt}`;

  // Create the HMAC Digest
  const digest = createHmac("sha256", identityKey).update(password).digest();

  // Returning a Buffer satisfies both Uint8Array requirements
  // and direct comparison logic in your Login function.
  return Buffer.from(digest);
};

/**
 * Generates cryptographic salts and hashes for a user password.
 *
 * Applies a 32-byte unique salt to the raw password input to generate
 * an encrypted Uint8Array for storage.
 *
 * @param props - Object containing username, email, and raw password string.
 * @returns Partial user object containing the new 32-byte hash and encrypted password.
 * @throws Error if props are undefined.
 */
export const SetPassword = async (props: Partial<IUser>): Promise<Partial<IUser>> => {
  if (!props) throw new ApiError(310, "Unauthorized access attempt; systems administrator notified.");

  const { username, email, password } = props;
  const hash = hashKey(32);
  const encrypt = hashPassword({ username, email, password, hash });

  return {
    hash,
    password: encrypt,
  };
};

export const PasswordReset = async (context = "User", credentials: Partial<IUser>, tempPass: string, log: ILogger) => {
  context = "Password.Reset";

```

```

const exists = await Fetch(context, credentials, log);

if (!exists) throw new ApiError(302, "Invalid user credentials");

// Generate HMAC using the passed-in temporary password
const newHash = hashPassword({ ...credentials, password: tempPass });
const [{ user }] = exists;
const reset = await State.Key(context, { status: "Reset" }, log);

await Update<IUser>(context, { user, password: newHash, state: reset }, { table: "user", keys: { user: "=" } }, log);
return true;
};

/**
 * Persists changes to an existing User in the local database.
 *
 * Performs a delta-check (via `isEqual`) to ensure only modified fields are
 * passed to the Update query. Resolves Role and State keys dynamically if
 * only titles or statuses are provided.
 *
 * @param props - The updated user attributes.
 * @param context - Log tracing context (defaults to "User").
 * @returns A standard API response indicating success or failure.
 */
export const Save = async (props: Partial<IUser>, context = "User", log: ILogger): Promise<TResponse> => {
  context = `${context}.Save`;

  if (!hasValues(props)) {
    return { success: false, code: 400, state: `null_query`, message: `[Error] ${context}:`, rows: 0, context };
  }

  const user = await Fetch(context, { username: props.username, email: props.email }, log);

  if (user) {
    const [current] = user;
    const role = props.role || (await Role.Key({ title: props.title })) || undefined;
    const state = props.state || (await State.Key({ status: props.status })) || undefined;
    const exists = true;

    const revised: Partial<IUser> = {
      user: props.user,
      role: role && isEqual(role, current.role!) ? undefined : role,
      state: state && isEqual(state, current.state!) ? undefined : state,
      image_url: props.image_url && props.image_url === current.image_url ? undefined : props.image_url,
    };
    return await Update<IUser>(context, revised, { table: `user`, keys: { user: "=" }, exists }, log);
  }
  return { success: false, code: 404, state: `not_found`, message: `[Error] ${context}:`, rows: 0, context: "User.Modify" };
};

/**
 * Provisions a new User in the system.
 *
 * Checks for existing identity, generates a 6-byte user key, and sets default
 * roles ("Viewer") and states ("Disabled") if not explicitly provided.
 *
 * @param props - New user registration data.
 * @param context - Log tracing context.
 * @returns The unique 6-byte user ID on success, or undefined if the user already exists.
 */
export const Add = async (context = "User", props: Partial<IUser>, log: ILogger): Promise<IUser["user"] | undefined> => {
  context = `${context}.Add`;

  const { username, email } = props;
  const user = await Key(context, { username, email }, log);

  if (user === undefined) {
    const { hash, password } = await SetPassword({ username, email, password: props.password });
    const add: Partial<IUser> = {
      user: hashKey(6),
      role: props.role || (await Role.Key({ title: props.title })) || (await Role.Key({ title: "Viewer" })),
      username,
      email,
      hash,
      password,
      state: props.state || (await State.Key({ status: props.status })) || (await State.Key<IAccess>({ status: "Disabled" })),
      image_url: props.image_url || `./images/user/no-image.png`,
    };
    const result = await Insert<IUser>(add, { table: `user`, context });

    return result.success ? add.user : undefined;
  } else return undefined;
};

/**
 * Retrieves a User's primary identifier based on seek parameters.
 *
 * Targets the `vw_users` database view to resolve the primary binary key.
 *
 * @param props - Search criteria (e.g., username, email).
 * @returns User ID or undefined if not found.
 */
export const Key = async (context = "User", props: Partial<IUser>, log: ILogger): Promise<IUser["user"] | undefined> => {
  if (!hasValues<Partial<IUser>>(props)) return undefined;

  context = `${context}.Key`;
  const result = await Select<IUser>(context, props, { table: `vw_users` }, log);
  return result.success && result.data?.length ? result.data[0].user : undefined;
};

/**
 * Fetches one or more user records from the system view.
 *
 * @param props - Query filters applied to the `vw_users` view.
 * @returns An array of partial user objects or undefined on query failure.
 */
export const Fetch = async (context = "User", props: Partial<IUser>, log: ILogger): Promise<Array<Partial<IUser>> | undefined> => {
  context = `${context}.Fetch`;
  const result = await Select<IUser>(context, props, { table: `vw_users` }, log);
  return result.success ? result.data : undefined;
};

/**
 * Authenticates a user attempt against stored cryptographic hashes.
 *
 * Validates the provided credentials against the stored salt and encrypted
 * password. Checks for account status; accounts not marked 'Enabled' are
 * rejected with specific status codes.
 *
 * @param props - Login credentials (username/email and raw password).

```

```

* @returns User object populated with error codes (301: Disabled, 302: Credentials) and status messages.
*/
export const Login = async (props: Partial<IUser>, log: ILogger): Promise<Partial<IUser>> => {
  const context = "User.Login";
  if (!hasValues) throw new ApiError(310, "Unauthorized access attempt; systems administrator notified.");

  const user = await Fetch(context, { username: props.username, email: props.email }, log);

  if (user) {
    const [login] = user;
    const { password, username, email, hash } = login;

    if (password instanceof Uint8Array) {
      const encrypt = Buffer.from(password);
      const key = hashPassword({ username, email, password: props.password, hash });

      if (isEqual(encrypt, key!)) {
        if (login.status === "Enabled") {
          Object.assign(login, { ...login, error: 0, message: "Connected" });
          return login;
        }
        Object.assign(login, { ...login, error: 301, message: `Your account is currently '${login.status}'. Contact your administrator.` });
        return login;
      }
      Object.assign(login, { ...login, error: 302, message: "Invalid username or password." });
      return login;
    }
    Object.assign(login, { ...login, error: 311, message: "Internal error. Contact your administrator." });
    return login;
  }

  const error: Partial<IUser> = {};
  Object.assign(error, { ...props, error: 302, message: "Invalid user credentials." });
  return error;
};

```